

1. Inverted Index & TF-IDF

Build: term $\rightarrow$ [(docID, tf), ...] with df per term

$w(t,d) = tf \times \log(N/df)$ [on cribsheet — use log table for log]

WHY: Fast lookup. IDF penalises common terms, rewards rare ones.

EXAM TRAPS:

- df = N $\rightarrow$ IDF = log(1) = 0 $\rightarrow$ term contributes ZERO (e.g. "interest")
- Count tf AFTER stopword removal — recount carefully from scratch
- Equal scores = equal rank (state both, e.g. Doc1=Doc3 > Doc2)

2. Vector Space Model — Cosine Similarity

$\cos(D,Q) = (D \cdot Q) / (|D| \times |Q|)$ [on cribsheet]

Steps:

- 1. Dot product: $\sum d_i \times q_i$ (only for terms in BOTH doc AND query)
- 2. $|D| = \sqrt{\text{sum of ALL weights}^2 \text{ in doc}}$, $|Q| = \sqrt{\text{sum of query weights}^2}$
- 3. Divide and round to 4dp
- Shorter docs score higher even if dot product is equal — cosine normalises length

3. Evaluation Metrics

Precision = $|Rel \cap Ret| / |Ret|$ Recall = $|Rel \cap Ret| / |Rel|$ F1 = $2PR / (P+R)$

AP = $(1/|R_total|) \times \text{sum } P(k)$ at each relevant rank [+ TOTAL not retrieved]

DCG = $rel + \sum rel / \log(i+1)$ nDCG = DCG / IDCG [log values given]

CRITICAL TRAPS:

- AP: always $\div$ TOTAL relevant — un-retrieved docs contribute 0 but still count
- From R-P graph: rank_k = k / P(at that recall point) — derive ranks this way
- Interpolated P(r) = MAX precision at recall $\geq$ r (look AHEAD in the graph)
- IDCG = ideal ranking (highest rel grades first) — then nDCG = DCG/IDCG

4. Heap's & Zipf's Laws

Heap: $V = K \times n^\beta$ ($\beta=0.5$ typical) $\rightarrow V = K \times \sqrt{n}$

Zipf: $r \times P(w|C) = A \rightarrow P(w)=A/1, P(w)=A/2, \dots$

- Heap steps: $n=\text{docs} \times \text{avgdl}$, find $K=V/\sqrt{n}$, $V_{\text{new}}=K \times \sqrt{n_{\text{new}}}$, diff=extra tokens
- Larger collections $\rightarrow$ fewer NEW terms per extra doc added (sublinear growth)
- Zipf: A = given_rank $\times$ given_prob. Top 4 words $\approx$ 20.8% of all tokens

5. Rocchio Relevance Feedback

$q_{\text{new}} = \alpha \cdot q + (\beta/|Dr|) \cdot \sum Dr - (\gamma/|Dn|) \cdot \sum Dn$ [on cribsheet]

Typical: $\alpha=1$ $\beta=0.75$ $\gamma=0.25$ — set NEGATIVE weights to 0

Steps:

- 1. q_{orig} = raw TF of query string (bargain=3, books=2, etc.)
- 2. Relevant centroid = average of relevant doc vectors ($\div |Dr|$)
- 3. Non-rel centroid = average of non-relevant doc vectors ($\div |Dn|$)
- 4. Apply formula term by term — drop any negatives (set to 0)

"Find Similar" feature: $\alpha=0$, $\beta=1$, $\gamma=0$ (clicked doc becomes new query)

6. Language Models — JM Smoothing

$P(w|d) = (1-\lambda) \cdot tf(w,d)/|d| + \lambda \cdot P(w|C)$ [on cribsheet]

Score(q|d) = PRODUCT of P(w|d) for every query term w

WHY: Zero-prob fix — missing terms (tf=0) still get $\lambda \cdot P(w|C) > 0$

- Rare terms (low P(w|C)) discriminate MORE — high tf of rare term wins
- $\lambda=0.5$ gives equal weight to doc frequency and collection frequency

Doc with higher tf of RARER term beats doc with higher tf of common terms

7. BIM — Binary Independence Model

$RSV = \sum \log[p(1-s) / s(1-p)]$ sum ONLY where $d=1$ (term present)

No relevance info: $p=0.5$ $s=df / N$

With relevance: $p=(r+0.5)/(R+1)$ $s=(df-r+0.5)/(N-R+1)$

TRAPS:

- Only sum for terms where $d=1$ (present in doc) AND in query
- Relevant doc WITHOUT query term $\rightarrow$ RSV=0 (big limitation of BIM)
- Non-relevant doc WITH term scores same as relevant doc — BIM cannot tell apart
- Use log — look up values from cribsheet log table

8. IR Model Assumptions — Always 4 marks!

- 1. Term independence (BoW): "New York" $\neq$ "New" + "York" (context lost)
- 2. Document independence: ignores diversity — 10 identical docs is useless
- 3. Static relevance: ignores time — 2019 COVID doc misleading in 2026
- 4. Topical = User relevance: ignores readability, credibility, authority

$\rightarrow$ For EACH: give the assumption AND a concrete real-world example!

9. HITS Algorithm — Hub & Authority

Hub: links TO many good authorities | **Authority:** linked BY many good hubs

$$a(p) = \sum h(q) \text{ for all } q \rightarrow p \quad h(p) = \sum a(q) \text{ for all } p \rightarrow q$$

Matrix method (exam: "using matrices, do NOT resolve equations"):

- 1. Build A: $A[i][j]=1$ if page i links TO page j
 - 2. Compute transpose A^T
 - 3. $M = A^T A \rightarrow$ write eigenvalue eqn: $M \cdot a = \lambda \cdot a$ (authority scores)
 - 4. $M = A A^T \rightarrow$ write eigenvalue eqn: $M \cdot h = \lambda \cdot h$ (hub scores)
 - 5. Diagonal $M[i][i]$ = in-degree of page $i \leftarrow$ use this to verify your matrix!
- Iterative: start $h=(1,1,\dots)$, update ALL a then ALL h , normalise by max each step

10. TAAT vs DAAT Index Traversal

TAAT: Process one FULL posting list, then next. Memory: $O(N)$ — bad for large N

DAAT: Process all terms for ONE doc, then move to next. Memory: $O(K)$ — good

MaxScore / WAND (dynamic pruning):

- Precompute $UB(t) = \text{max score contribution of term } t \text{ to any doc in its list}$
- Skip doc if $\text{partial_score} + \text{remaining_UBs} < \text{current threshold } \theta \rightarrow \text{saves work}$

11. Index Compression — Lossy vs Lossless

Lossless: No info lost. d-gaps, VByte, Gamma codes, PForDelta

$\rightarrow$ Measure: compression ratio + decompression speed. MAP unchanged.

Lossy: Info discarded. Static pruning, stopwords removal, quantisation

$\rightarrow$ Measure: compression ratio AND MAP/nDCG before vs after (effectiveness measure)

- Acceptable if MAP drops $< 1\text{--}2\%$ with significant index size reduction

12. Exam Strategy — 1h 30min, 60 marks

- Q1 (~25 min, ~20 marks): Inverted index $\rightarrow$ TF-IDF $\rightarrow$ Cosine $\rightarrow$ P/R/AP/nDCG
- Q2 (~25 min, ~20 marks): Heap's $\rightarrow$ Rocchio $\rightarrow$ JM Smoothing $\rightarrow$ Conceptual Q
- Q3 (~25 min, ~20 marks): Neural IR $\rightarrow$ PyTerrier $\rightarrow$ 4 Assumptions or HITS

■ **ALWAYS: write formula $\rightarrow$ substitute values $\rightarrow$ compute $\rightarrow$ state result = partial marks**

- Check: IDF=0 trap, $AP \div \text{total_relevant}$, normalise HITS each iteration

13. Neural IR — Architecture Trade-offs

Bi-Encoder: Encodes q and d SEPARATELY. Pre-index docs offline $\rightarrow$ fast ANN search

- Use as FIRST STAGE — finds semantic matches (handles vocabulary mismatch)

Cross-Encoder: Encodes (q,d) JOINTLY — full interaction, best quality but slow

- Cannot pre-compute $\rightarrow$ use as RE-RANKER on top-100 only, never first stage

ColBERT: Token-level late interaction — best of both (pre-encode docs)

$$\text{MaxSim}(Q,D) = \sum \max \text{sim}(q_i, d_j) \text{ over all } q_i \in Q \quad [\text{on cribsheet}]$$

■ **NaN p-value: re-ranker same 1000 candidates $\rightarrow$ Recall@1000 identical $\rightarrow$ not significant**

14. PyTerrier Pipelines — Up to 16 marks!

Operators: `>>` Then `%n` Top-n cutoff | Union & Intersect

PRF pipeline: `bm25 % 3 >> rewriter1(index) >> bm25`

- `rewriter1`: sees top-3 DOCS + index. Uses Bo1 / RM3 (pseudo-relevance feedback)
- `rewriter2 >> bm25`: sees ONLY the query string. Uses thesaurus / Word2Vec

Experiment template:

```
pt.Experiment([base, system], topics, qrels,
              eval_metrics=["ndcg_cut_10", "map", "recall_1000"], baseline=0)
```

- `baseline=0` enables auto paired t-test. Justify: NDCG@10 = user-facing quality

15. Dense Retrieval + Passage Aggregation

■ **PROBLEM: BERT 512-token limit; single vector loses long-document context**

FIX: Split into overlapping passages, encode each independently

MaxPassage: score = BEST single passage score (one strong section proves relevance)

SumPassage: score = SUM of all passage scores (evidence spread throughout)

- Content SPREAD through doc? $\rightarrow$ Use SumPassage (accumulates all evidence)
- ONE highly relevant section? $\rightarrow$ MaxPassage is sufficient